Trabajo Práctico Final Estrategia de Testing
Aplicación: Clínica



Materia: Calidad de Software A

Docentes: Adolfo Tomas Spinelli, Jessica Elizabeth Miguel , Paula Bonifazi

Integrantes: Bellone Martin, Garcia Di Martino Laureano, Sorrenti Enzo, Zuviria Nicolas

Año: 2026

Facultad de Ingenieria, UNMDP



Índice

Sección	Página
Introducción	4
1.1 Objetivo	4
1.2 Descripción del sistema en cuestión	4
1.3 Funcionalidades a probar	4
1.4 Fuera del alcance	4
1.5 Referencias	4
Módulo 1: Testing de persistencia de los datos de asociados	6
AsociadoDTO	6
Resultados de Ejecución y Detección de Defectos (AsociadoDTOTest)	7
Conclusiones de las Pruebas Unitarias sobre AsociadoDTO	7
Testing de Integración: BaseDeDatos (Singleton y JDBC)	8
Estrategia de Prueba y Configuración del Entorno (@Before y @After)	8
Planteo de Escenarios de Integración	8
Clases de Equivalencia	9
Casos de prueba (Escenarios 1 a 4)	9
Conclusiones del Testing de Integración sobre BaseDeDatos	13
Testing de Integración: DataAccessObject (Mapeo ORM y Operaciones CRUD)	14
Estrategia de Prueba y Alcance	14
Planteo de Escenarios de Prueba	14
Clases de Equivalencia y Valores Límite Aplicados	14
Casos de prueba (Escenarios 1 a 4)	15
Conclusiones del Testing de Integración sobre DataAccessObject	18
Módulo 2: Testing de la máquina de estados	20
Objetivo y Alcance de la Prueba	20
Estrategia de Testing y Uso de Dobles de Prueba (Mocks)	20
Diseño de Casos de Prueba (Particiones de Equivalencia)	20
Detalle de Escenarios Implementados en testAmbulancia	21
Batería de Casos de Prueba (testAmbulancia)	21
Resultados de Ejecución y Detección de Defectos (AmbulanciaTest)	23
Análisis del Defecto Detectado	24
Conclusiones del Módulo de Máquina de Estados	24
Módulo 3: Testing de las funcionalidades de concurrencia	24
Tabla de Particiones de Equivalencia	26
Batería de Casos de Prueba	27
Resultados de Ejecución y Detección de Defectos (AmbulanciaConcurrenciaTest)	27
Conclusiones del Módulo de Concurrencia	28



Conclusión General del Informe 29
Tabla global de resultados 29
Tabla consolidada de defectos 30
Anexo 31

Introduccion:

1.1 Objetivo.

El presente informe documenta la estrategia de testing aplicada sobre el sistema de gestión *Clínica*, en el marco del Trabajo Práctico Final de Calidad de Software. Su propósito es diseñar, ejecutar y documentar pruebas en dos niveles: testing unitario sobre componentes aislados y testing de integración sobre componentes acoplados a recursos externos (base de datos) e hilos de ejecución. Se busca demostrar que las suites desarrolladas detectan los defectos presentes en el código provisto.

1.2 Descripción del sistema en cuestión.

El sistema a testear es un software de gestión de una pequeña clínica privada el cual cuenta con los siguientes módulos solicitados: Facturación al Paciente, Reportes de Actividad de los Médicos (con suma de honorarios), Resolución de Conflictos de Sala de Espera, gestión de Asociados, simulación de uso de una Ambulancia, persistencia de datos y una interfaz gráfica de usuario. Sin embargo, los módulos y funcionalidades que son alcanzadas por este informe y por las pruebas realizadas son la persistencia de los datos de Asociados, la máquina de estados y la simulación de la clase Ambulancia.

1.3 Funcionalidades a probar:

1	Persistencia de los datos de asociados	DataAccesObject, AsociadoDTO, BaseDeDatos
2	Máquina de estados	EnTallerState, RegresandoDelTallerState, DisponibleState, AtendiendoEnDomicilioState, TrasladandoPacienteAClinicaState, RegresandoClinicaSinPacienteState
3	Concurrencia	Ambulancia, RetornoAutomatico, Asociado, Operario

1.4 Fuera del alcance. No se prueban los módulos de Facturación al Paciente, Reportes de Actividad de los Médicos, Resolución de Conflictos de Sala de Espera ni la interfaz gráfica de usuario.

1.5 Referencias.

- Enunciado del Trabajo Práctico Final “Estrategia de Testing – Aplicación: Clínica”, provisto por la cátedra.
- Especificación de Requisitos de Software (SRS) para el Módulo de Persistencia.



- Código fuente del sistema provisto por la cátedra.
- Documentación oficial de JUnit y Mockito.



Modulo 1: Testing de persistencia de los datos de asociados

El módulo de persistencia de los datos de asociados cuenta con las siguientes clases: DataAccessObject, AsociadoDTO, BaseDeDatos. Estas clases se organizan en capas: cada una tiene una responsabilidad propia.

- BaseDeDatos maneja la conexión con MySQL mediante JDBC y ejecuta las sentencias SQL. Implementa el patrón Singleton, de modo que todo el sistema comparte una sola conexión.
- AsociadoDTO es un objeto de transferencia de datos, sin lógica, que guarda los datos de un asociado tal como están en la tabla (nombre, apellido, DNI, domicilio y teléfono).
- DataAccessObject (patrón DAO) ofrece al resto del sistema las operaciones de alta, baja, modificación y consulta de asociados.

En cuanto a la relación entre ellas, el DAO tiene una referencia a la instancia de BaseDeDatos, a la que le delega la ejecución de las consultas. Además, convierte cada Asociado del dominio en un AsociadoDTO antes de enviarlo a la base, y cada fila que devuelve una consulta en un AsociadoDTO. Así, AsociadoDTO lleva los datos entre las capas y la lógica de negocio queda separada de los detalles de la persistencia.

El testing de este módulo se comenzó realizando pruebas unitarias para la clase AsociadoDTO.

AsociadoDTO

Escenarios: En las pruebas de unidad de esta clase no se presentan escenarios.

nombre	nombre<> null	-
apellido	apellido <> null	-
dni	dni <> null	-
ciudad	ciudad <> null	-
calle	calle <> null	-
numero	numero <> null	-
teléfono	teléfono <> null	-

En la tabla de particiones no se consideran clases invalidas ya que los caoss de fallo son precondiciones, por ese motivo no se testean

Batería de Casos de Prueba (AsociadoDTO):

CP-DTO-1	"Enzo"	Asociado.getNombre() = "Enzo"	1	Se setea correctamente
CP-DTO-2	“Sorrenti”	Asociado.getApellido() = “Sorrenti”	1	Se setea correctamente



CP-DTO-3	“40123456”	Asociado.getDni() = “40123456”	1	Se setea correctamente
CP-DTO-4	“Mar del Plata”	Asociado.getCiudad() = “Mar del Plata”	1	Se setea correctamente
CP-DTO-5	“Av. Colón”	Asociado.getCalle() = “Av. Colón”	1	Se setea correctamente
CP-DTO-6	2500	Asociado.getNumero() = 2500	1	Se setea correctamente
CP-DTO-6.1	0	Asociado.getNumero() = 0	1	AssertionError
CP-DTO-7	“2235123456”	Asociado.getTelefono() = “2235123456”	1	Se setea correctamente

Resultados de Ejecución y Detección de Defectos (AsociadoDTOTest)

Al ejecutar la suite de pruebas unitarias AsociadoDTOTest (compuesta por 8 casos de prueba atómicos ejecutados de forma estrictamente aislada, sin dependencias ni conexión al motor MySQL), se obtuvo el siguiente balance de resultados (ver figura 1 del anexo):
Tests ejecutados: 8
Pruebas exitosas (Estado Verde): 7 (testSetNombre(), testSetApellido(), testSetDni(), testSetCiudad(), testSetCalle(), testSetTelefono() y testSetNumero()).
Pruebas fallidas (Estado Rojo): 1 (testSetNumeroLimite()).

Detalle del defecto detectado:
Caso de prueba fallido: testSetNumeroLimite().
Traza del fallo: `java.lang.AssertionError: 'setNumero(0) lanzo AssertionError'.`
Análisis de causa raíz: Según el contrato y la especificación de dominio para la altura del domicilio (numero), la clase de equivalencia válida abarca los números enteros mayores o iguales a cero (numero >= 0). Mientras que el caso de prueba nominal con un entero positivo (testSetNumero()) finalizó exitosamente en verde, al evaluar específicamente el valor límite inferior (0) mediante testSetNumeroLimite(), el método setNumero(0) arrojó un java.lang.AssertionError. Esto evidencia un defecto de borde en la validación interna de integridad del *setter* dentro de AsociadoDTO, donde la condición fue codificada con una desigualdad estricta (> 0) en lugar de inclusiva (>= 0), rechazando erróneamente el valor frontera 0. (La captura de ejecución en el Test Runner se adjunta en el Apéndice).
Conclusiones de las Pruebas Unitarias sobre AsociadoDTO
A partir del diseño y ejecución de la suite AsociadoDTOTest, se desprenden las siguientes conclusiones técnicas:
Importancia del Análisis de Valores Límite: Quedó demostrado que cubrir únicamente valores representativos dentro de una partición de equivalencia no es suficiente para garantizar la ausencia de defectos. El método testSetNumero() superó la prueba con un valor típico de la clase válida (> 0), pero fue la separación de un caso de prueba dedicado exclusivamente a la frontera (testSetNumeroLimite() con entrada 0) lo que permitió exponer el fallo en la condición del assert interno de setNumero.
Validación de Aislamiento Atómico: La totalidad de los *setters* y *getters* de tipo String (nombre, apellido, dni, ciudad, calle, telefono) cumplen correctamente con su responsabilidad de



almacenamiento y recuperación en memoria. Realizar esta verificación de manera unitaria permitió certificar la estructura interna del objeto de transferencia antes de acoplarlo con el DataAccessObject y las consultas SQL en las pruebas de integración.

Impacto del contrato de setNumero hacia las capas superiores: La presencia de esta validación por assert en setNumero no solo afecta la asignación directa en memoria, sino que condiciona la reconstrucción de objetos desde la base de datos (como se demuestra posteriormente en las pruebas de integración de DataAccessObjectTest al leer registros con alturas fuera de rango desde MySQL).

Testing de Integración: BaseDeDatos (Singleton y JDBC)

Estrategia de Prueba y Configuración del Entorno (@Before y @After)
A diferencia de las pruebas unitarias de AsociadoDTO, esta suite evalúa la integración real entre el código Java, el driver JDBC y el motor de base de datos relacional MySQL (esquema Grupo_7, tabla Asociados).
Para garantizar que las pruebas sean independientes, repetibles y determinísticas, el método setUp() (@Before) realiza dos acciones críticas antes de cada test:
Verificación y recuperación de la instancia Singleton: Obtiene la instancia mediante BaseDeDatos.getInstancia(). Si por alguna prueba previa la conexión subyacente quedó cerrada (isClosed()), utiliza reflexión (Field.setAccessible(true)) para reiniciar el atributo estático instancia a null y forzar una reconexión limpia.
Restauración del estado inicial (iniciarBD()): Invoca baseDeDatos.iniciarBD() antes de cada caso de prueba, asegurando que la tabla Asociados se recree desde cero con sus 20 registros semilla originales, evitando que los INSERT, UPDATE o DELETE de un test contaminen la ejecución del siguiente.

Planteo de Escenarios de Integración

A partir de las responsabilidades de la clase BaseDeDatos, se estructuraron 4 escenarios de prueba:

Escenario 1: Verificación del Patrón Singleton y Conexión JDBC. Evalúa que BaseDeDatos.getInstancia() retorne siempre la misma referencia en memoria y mantenga una conexión JDBC activa y abierta contra MySQL.

Escenario 2: Inicialización Automática de Esquema y Datos Semilla (iniciarBD). Verifica la creación efectiva de la tabla Asociados, la cantidad exacta de registros iniciales (20), los valores frontera del lote semilla (id = 1 e id = 20), la idempotencia del método de reinicio y la consistencia de formato en las columnas cargadas por defecto (ciudad y telefono).

Escenario 3: Operaciones CRUD Parametrizadas (PreparedStatement). Evalúa la ejecución de altas (ejecutarInsert), bajas (ejecutarDelete), modificaciones (ejecutarActualizacion) y consultas (ejecutarConsulta) enviando objetos AsociadoDTO y parámetros escalares a la base de datos.

Escenario 4: Manejo de Excepciones SQL, Restricciones de Integridad y Precondiciones. Evalúa la respuesta del componente ante violaciones de restricciones de base de datos (UNIQUE, longitud de VARCHAR(10), sintaxis SQL), violaciones de contratos por valores null (AssertionError), el uso del valor centinela id = -1 y la inserción de un AsociadoDTO con estado inválido creado por constructor.



Clases de Equivalencia:

Para las operaciones que reciben parámetros en BaseDeDatos, se definieron las siguientes particiones de equivalencia:

ejecutarConsulta(sql, id)	CV1: sql SELECT válido y id == -1 (consulta general sin filtro). CV2: sql con parámetro ? e id >= 1 existente (1..20). CV3: sql con ? e id > 20 inexistente (ej. 999, devuelve ResultSet vacío).	CI1: sql == null (AssertionError). CI2: sql con tabla inexistente o sintaxis inválida (SQLException). CI3: sql con ? pero pasando el centinela id == -1 (no bindea el parámetro).
ejecutarInsert(sql, dto)	CV1: sql INSERT válido y AsociadoDTO completo con dni único (dni > 10 caracteres) y numero > 0.	CI1: sql == null o dto == null (AssertionError). CI2: dto con dni ya existente en la tabla (SQLException por UNIQUE). CI3: dto con dni (SQLException por truncamiento). CI4: dto instanciado con atributos null y numero < 0 (-10).
ejecutarActualizacion(sql, dto)	CV1: sql UPDATE válido y dto con datos modificados para actualizar el registro objetivo sin alterar otros registros.	CI1: sql == null o dto == null (AssertionError).
ejecutarDelete(sql, dni)	CV1: sql DELETE válido y dni existente en la tabla ("12345678").	CI1: sql == null o dni == null (AssertionError).

Escenario 1: Conexión y Patrón Singleton

Precondición: Motor MySQL activo y setUp() ejecutado.

CP-BD-1.1	Invocar dos veces BaseDeDatos.getInstancia() y consultar getConexion().isClosed()	Referencias idénticas (assertSame), conexión no nula y abierta (isClosed() == false).	Misma instancia y conexión abierta (PASA)

Escenario 2: Inicialización Automática de Tablas y Datos Semilla (iniciarBD)
Precondición: Ejecución previa de baseDeDatos.iniciarBD() en @Before.

CP-BD-2.1	Ejecutar query "SHOW TABLES;" sobre la conexión activa	Existencia de la tabla llamada "Asociados" en el esquema Grupo_7.	Tabla "Asociados" encontrada (PASA)
CP-BD-2.2	ejecutarConsulta("SELECT * FROM Asociados;", -1)	El ResultSet contiene exactamente 20 filas.	20 filas contabilizadas (PASA)
CP-BD-2.3	Consultar valores límite id = 1 e id = 20	id=1: "Juan", "State", "12345678", "Buenos Aires", 123, "Mar del Plata", "2235551234". id=20: "Adrian", "Composite", "28293031".	Datos coinciden exactamente (PASA)
CP-BD-2.4	Inspeccionar columna ciudad en los 20 registros semilla	Todos los registros tienen "Mar del Plata" sin espacios iniciales.	El registro id = 4 tiene " Mar del Plata" con un espacio inicial (FALLA)
CP-BD-2.5	Inspeccionar longitud de columna telefono en los 20 registros semilla	Todos los teléfonos semilla tienen 10 dígitos (telefono.length() == 10).	Los registros id = 12 ("155889900") e id = 17 ("155445566") tienen 9 dígitos (FALLA)
CP-BD-2.6	Borrar DNI "12345678", insertar un DTO temporal y volver a llamar a iniciarBD()	La tabla se reinicia y vuelve a tener exactamente los 20	20 registros restaurados (PASA)



		asociados semilla originales.	

Escenario 3: Operaciones CRUD Parametrizadas sobre BaseDeDatos
Precondición: Tabla Asociados inicializada con los 20 registros semilla (id 1 al 20).

CP-BD-3.1	ejecutarInsert con AsociadoDTO("Enzo", "Sorrenti", "40506070", "Mar del Plata", "Constitucion", 5000, "2234445566")	Se inserta el registro con id = 21 y todos sus campos coinciden al consultarlo.	Registro id = 21 insertado y verificado (PASA)
CP-BD-3.2	ejecutarConsulta("SELECT * FROM Asociados WHERE id = ?;", 999)	Retorna un ResultSet no nulo y vacío (rs.next() == false).	ResultSet vacío (PASA)
CP-BD-3.3	ejecutarActualizacion(updateSQL, dtoActualizado) y luego consultar el registro id = 1	El método debería permitir indicar el ID a modificar sin sobrescribir por defecto al asociado id = 1 ("Juan").	Sobrescribe al registro id = 1 con "MariaModificada" porque tiene hardcodeado stmt.setInt(8, 1) (FALLA)
CP-BD-3.4	ejecutarDelete("DELETE FROM Asociados WHERE dni = ?;", "12345678")	El registro con id = 1 (DNI "12345678") ya no existe al consultarlo (rs.next() == false).	Registro eliminado correctamente (PASA)

Escenario 4: Captura de Excepciones SQL y Validación de Precondiciones
Precondición: Tabla Asociados inicializada con los 20 registros semilla.

CP-BD-4.1	ejecutarInsert con dni = "12345678" (ya existente en id = 1)	Lanza SQLException por violación de restricción UNIQUE en la columna DNI.	Lanza SQLException (PASA)



CP-BD-4.2	ejecutarInsert con dni = "123456789012345" (15 caracteres > VARCHAR(10))	Lanza SQLException por truncamiento de datos (*Data truncation*).	Lanza SQLException (PASA)
CP-BD-4.3	ejecutarConsulta("SELECT * FROM TablaInexistente WHERE id = ?;", 1)	Lanza SQLException porque la tabla no existe en el esquema.	Lanza SQLException (PASA)
CP-BD-4.4	Invocar ejecutarConsulta, ejecutarInsert, ejecutarActualizacion y ejecutarDelete con argumentos null	Todos los llamados lanzan AssertionError al violar las precondiciones assert != null.	Lanza AssertionError en los 7 casos (PASA)
CP-BD-4.5	ejecutarConsulta("SELECT * FROM Asociados WHERE id = ?;", -1)	Lanza SQLException porque la condición interna if (id != -1) omite el stmt.setInt(1, id) dejando el parámetro ? sin valor.	Lanza SQLException (PASA)
CP-BD-4.6	ejecutarInsertconnewAsociadoDTO(null, null, null, null, null, -10, null)	Debería lanzar excepción (AssertionError o SQLException) impidiendo persistir campos nulos y numero = -10.	Se inserta en MySQL sin lanzar ninguna excepción porque ni el constructor de AsociadoDTO ni el DDL de la tabla validan

			NOT NULL o CHECK (numero >= 0) (FALLA)

Conclusiones del Testing de Integración sobre BaseDeDatos

La ejecución de los 17 casos de prueba de la suite BaseDeDatosTest contra el motor MySQL permitió validar la comunicación real por JDBC y exponer defectos estructurales y de datos que permanecían invisibles en las pruebas unitarias aisladas:

Correcta implementación del Patrón Singleton y Operaciones Básicas (Pruebas en Verde):

Se comprobó que BaseDeDatos.getInstancia() garantiza una única instancia compartida con una conexión JDBC activa, y que el método iniciarBD() es idempotente, recreando correctamente la tabla Asociados y poblando los 20 registros semilla iniciales.

Las operaciones parametrizadas de inserción (ejecutarInsert), eliminación por DNI (ejecutarDelete) y consulta por clave primaria (ejecutarConsulta) funcionan correctamente para casos válidos, mientras que el motor MySQL y las precondiciones (assert) rechazan adecuadamente las claves duplicadas (UNIQUE en DNI), los excesos de longitud (VARCHAR(10)) y los argumentos null.

Detección de Inconsistencias en los Datos Semilla (iniciarBD - Pruebas en Rojo):

Formato de ciudad (CP-BD-2.4): El registro semilla con id = 4 (*Jose Concurrencia*) se inserta con un espacio en blanco inicial (" Mar del Plata"), rompiendo la uniformidad de datos frente a búsquedas exactas por ciudad.

Longitud de telefono (CP-BD-2.5): Los registros con id = 12 ("155889900") e id = 17 ("155445566") poseen teléfonos de 9 dígitos en lugar de los 10 dígitos estándar del resto del lote semilla.

Defecto Crítico en la Operación de Actualización (ejecutarActualizacion - Prueba en Rojo):

Índice de registro hardcodeado (CP-BD-3.3): Se detectó un fallo severo en BaseDeDatos.ejecutarActualizacion(String, AsociadoDTO). El método no recibe por parámetro el id del asociado que se desea modificar y, en su lugar, asigna de forma estática stmt.setInt(8, 1) en el PreparedStatement. Como consecuencia, cualquier intento de actualización desde el sistema sobrescribe invariablemente al primer asociado de la tabla (id = 1, *Juan State*), corrompiendo la integridad de los datos almacenados.

Brecha de Integridad entre AsociadoDTO y el Esquema DDL (Prueba en Rojo):

Falta de validación en constructor y en tabla SQL (CP-BD-4.6): Si bien los *setters* de AsociadoDTO poseen precondiciones (assert), su constructor parametrizado asigna los atributos directamente sin validarlos, y la sentencia CREATE TABLE Asociados dentro de iniciarBD() carece de restricciones NOT NULL y CHECK (numero >= 0). Esto permite que un AsociadoDTO instanciado con campos null y altura de calle negativa (-10) atraviese ambas capas y se persista en MySQL sin lanzar AssertionError ni SQLException.

Acoplamiento por valor centinela (id = -1 en CP-BD-4.5): El método ejecutarConsulta utiliza id != -1 para decidir si asigna o no el parámetro del PreparedStatement. Si se invoca una consulta parametrizada (WHERE id = ?) pasando -1, el parámetro queda sin ligar y provoca una SQLException en tiempo de ejecución.



Testing de Integración: DataAccessObject (Mapeo ORM y Operaciones CRUD)

Estrategia de Prueba y Alcance

La suite DataAccessObjectTest evalúa la capa de acceso a datos (DataAccessObject), la cual actúa como intermediaria entre las entidades del modelo de dominio (Asociado, Domicilio), el objeto de transferencia (AsociadoDTO) y el componente de conexión (BaseDeDatos).
Al igual que en BaseDeDatosTest, el método setUp() (@Before) garantiza la apertura de la conexión Singleton, ejecuta baseDeDatos.iniciarBD() para restaurar los 20 registros semilla antes de cada prueba e instancia un nuevo objeto DataAccessObject.

Planteo de Escenarios de Prueba

Se diseñaron 4 escenarios de integración que cubren todas las responsabilidades del DAO:

Escenario 1: Mapeo de Entidades y Construcción de DTOs (CrearDTO). Verifica las dos sobrecargas del método CrearDTO (a partir de una instancia de Asociado y a partir de parámetros escalares leídos de un ResultSet), así como su comportamiento ante referencias internas nulas (Domicilio == null).

Escenario 2: Operaciones de Alta y Consultas de Lectura (guardar, cargarBD, cargarConLimite, cargar). Evalúa el ciclo completo de persistencia de un nuevo asociado, la recuperación masiva de los 20 registros semilla, la paginación/recorte mediante límites y la búsqueda puntual por clave primaria (id).

Escenario 3: Operaciones de Modificación y Baja (actualizar, eliminar). Verifica el impacto en la base de datos al modificar los atributos de un asociado existente o recién creado, y la baja física de registros filtrando por DNI (tanto existente como inexistente).

Escenario 4: Excepciones de SQL, Precondiciones y Corrupción de Datos. Evalúa el rechazo de claves duplicadas (DNI), desbordamiento de columnas (VARCHAR(15) en telefono), violaciones de precondiciones (assert), persistencia de cadenas vacías ("") y el efecto cascada que produce un registro corrupto en la base de datos al intentar ejecutar cargarBD().

Clases de Equivalencia y Valores Límite Aplicados

En la siguiente tabla se sistematizan las particiones de equivalencia y los valores límite definidos para los métodos de DataAccessObject:

Método Evaluado	Clases Válidas (CV)	Clases Inválidas (CI)	Valores Límite / Frontera
CrearDTO(...)	CV1: Asociado no nulo con Domicilio válido. CV2: Parámetros escalares válidos (numero >= 0 y strings no nulos).	CI1: asociado == null (AssertionError). CI2: Asociado con domicilio == null (NullPointerException). CI3: Parámetro escalar numero < 0 (AssertionError en setNumero).	numero = 0 / numero = -5 (frontera de precondición al reconstruir el DTO desde la BD).



cargarConLimite(limite)	CV1: 1 <= limite <= 20 (retorna exactamente limite asociados). CV2: limite > 20 (supera el total en BD; retorna el tope disponible de 20 asociados).	CI1: limite <= 0 (AssertionError por precondición limite > 0).	Límite inválido: 0 y -5. Límite inferior válido: 1. Valor nominal: 5. Frontera de tabla: 20 (exacto) y 25 (exceso).
cargar(id)	CV1: id >= 1 existente en BD (1..20, retorna el AsociadoDTO). CV2: id > 20 inexistente en BD (ej. 999, debería retornar null).	CI1: id <= 0 (AssertionError por precondición id > 0).	Límites inválidos: 0 y -1. Límite inferior válido: id = 1. Valor fuera de rango: id = 999.
guardar(asociado)	CV1: Asociado válido con dni único (dni > 10 caracteres), teléfono 15 caracteres y campos con contenido.	CI1: asociado == null (AssertionError). CI2: dni duplicado en BD (SQLException). CI3: teléfono 15 caracteres (SQLException). CI4: Campos vacíos "" (AssertionError esperado).	Longitud telefono: 10 chars (válido) vs. 20 chars (excede VARCHAR(15)). Longitud mínima de Strings: 0 caracteres ("").
actualizar(asociado)	CV1: Asociado existente o nuevo con atributos modificados.	CI1: asociado == null (AssertionError).	Afectación colateral sobre el registro frontera id = 1 (*Juan State*).
eliminar(dni)	CV1: dni no vacío existente en BD ("12345678", reduce total a 19). CV2: dni no vacío inexistente ("99999999", mantiene total en 20).	CI1: dni == null (AssertionError). CI2: dni vacío "" (AssertionError).	Longitud de dni: 0 caracteres ("", límite inválido).

Escenario 1: Mapeo Objeto-Relacional (CrearDTO)
Precondición: Instancia de DataAccessObject inicializada.

ID	Entrada / Acción	Salida Esperada	Salida Obtenida
CP-DAO-1.1	dao.CrearDTO(asociado) con "Laureano", "Garcia", "42111222", "2235998877",	Retorna AsociadoDTO con los 7 atributos mapeados sin pérdida de datos.	Mapeo exacto (PASA)



	Domicilio("Av. Colon", 1500, "Mar del Plata")		
CP-DAO-1.2	dao.CrearDTO(10, "Lucas", "Paniagua", "41222333", "San Juan", 2400, "Mar del Plata", "2236123456")	Retorna AsociadoDTO con todos los campos asignados correctamente.	Mapeo exacto (PASA)
CP-DAO-1.3	dao.CrearDTO(asociadoSinDomicilio) donde domicilio == null	Lanza NullPointerException al invocar asociado.getDomicilio().getCalle().	Lanza NullPointerException (PASA)

Escenario 2: Operaciones de Alta y Lectura (guardar, cargarBD, cargarConLimite, cargar)
Precondición: Base de datos restaurada con los 20 asociados semilla (id 1 al 20).

ID	Entrada / Acción	Salida Esperada	Salida Obtenida
CP-DAO-2.1	dao.guardar(nuevo) ("Nicolas", "Zuviria", "43999888") y posterior dao.cargarBD()	La lista aumenta a 21 registros y el elemento en índice 20 coincide con el asociado guardado.	Persistido y recuperado en posición 21 (PASA)
CP-DAO-2.2	Crear Asociado(dtoEntrada) ("Luciano", "Vega", "45123456"), llamar a dao.guardar(...) y dao.cargarBD()	Conversión bidireccional DTO -> Asociado -> DTO -> MySQL exitosa; lista de tamaño 21.	Flujo completo exitoso (PASA)
CP-DAO-2.3	dao.cargarBD() sobre el estado inicial	Retorna ArrayList con 20 elementos; el primero es "Juan" ("12345678") y el último "Adrian" ("28293031").	Retorna los 20 asociados semilla (PASA)
CP-DAO-2.4	Invocar dao.cargarConLimite(n) con n = 1, 5, 20 y 25	Para 1, 5 y 20 retorna 1, 5 y 20 asociados respectivamente; para 25 retorna el tope de 20 existentes.	Tamaños 1, 5, 20 y 20 verificados (PASA)
CP-DAO-2.5	dao.cargar(1)	Retorna el AsociadoDTO de "Juan State" (dni = "12345678").	Lanza SQLException: Before start of result set porque cargar(id) no llama a resultado.next() antes de leer las columnas (FALLA)
CP-DAO-2.6	dao.cargar(999)	Retorna null al no existir el registro id = 999.	Lanza SQLException: Before start of result set por no verificar if (resultado.next())



			(FALLA)

Escenario 3: Operaciones de Actualización y Baja (actualizar, eliminar)
Precondición: Base de datos restaurada con los 20 asociados semilla.

ID	Entrada / Acción	Salida Esperada	Salida Obtenida
CP-DAO-3.1	dao.actualizar(mariaModificada) (correspondiente a id = 2, dni = "87654321")	Actualiza el apellido, domicilio y teléfono del registro con dni = "87654321".	Lanza SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'Asociados.DNI' porque intenta pisar el id = 1 con el DNI del id = 2 (FALLA)
CP-DAO-3.2	Guardar a "Martin Bellone" (id = 21) y luego invocar dao.actualizar(modificado) con un nuevo DNI ("44999888")	Se actualiza el asociado id = 21 y el registro id = 1 (*Juan State*) permanece intacto.	El registro id = 1 (*Juan State*) es sobrescrito con los datos de "MartinMod" debido al stmt.setInt(8, 1) hardcodeado (FALLA)
CP-DAO-3.3	dao.eliminar("12345678") y luego dao.cargarBD()	La cantidad total disminuye a 19 y el DNI "12345678" ya no se encuentra en la lista.	Asociado eliminado; quedan 19 registros (PASA)
CP-DAO-3.4	dao.eliminar("99999999") y luego dao.cargarBD()	No se elimina ninguna fila; se mantienen los 20 registros semilla.	Se mantienen los 20 registros (PASA)

Escenario 4: Excepciones de SQL y Validaciones de Integridad
Precondición: Base de datos restaurada con los 20 asociados semilla.

ID Caso	Entrada / Acción	Salida Esperada	Salida Obtenida
CP-DAO-4.1	dao.guardar(duplicado) con dni = "12345678"	Lanza SQLException por restricción UNIQUE en la columna DNI.	Lanza SQLException (PASA)
CP-DAO-4.2	dao.guardar(telefonoLargo) con telefono de 20 caracteres (> VARCHAR(15))	Lanza SQLException por truncamiento de datos (*Data truncation*).	Lanza SQLException (PASA)
CP-DAO-4.3	Llamar a guardar(null), actualizar(null),	Lanza AssertionError en	Lanza AssertionError en



	CrearDTO(null), cargar(0), cargar(-1), cargarConLimite(0), cargarConLimite(-5), eliminar("") y eliminar(null)	las 9 invocaciones al incumplir las precondiciones assert.	todos los casos (PASA)
CP-DAO-4.4	dao.guardar(new Asociado("", "", "", "", new Domicilio("", 1, "")))	Debería lanzar AssertionError rechazando un asociado sin nombre, apellido ni DNI.	Se inserta en MySQL sin lanzar excepción porque ninguna capa valida cadenas vacías "" en el alta (FALLA)
CP-DAO-4.5	Insertar AsociadoDTO con numero = -5 por constructor y luego invocar dao.cargarBD()	El sistema no debería haber permitido el insert inválido o cargarBD() debería poder manejarlo sin colapsar.	dao.cargarBD() colapsa lanzando AssertionError cuando CrearDTO invoca setNumero(-5) (FALLA)

Conclusiones del Testing de Integración sobre DataAccessObject

El análisis de los 18 casos de prueba ejecutados en DataAccessObjectTest (12 en Verde y 6 en Rojo, ver figura 4 en anexo) permitió evaluar la integración de las tres clases del módulo de persistencia y documentar los siguientes hallazgos críticos:

Correcto funcionamiento de las operaciones de Alta, Baja, Listado y Precondiciones (Verde):

El mapeo de objetos mediante CrearDTO, el alta de asociados válidos (guardar), la eliminación por DNI (eliminar), la carga masiva (cargarBD) y la carga acotada (cargarConLimite evaluada en sus valores límite 1, 20 y 25) funcionan conforme a la especificación. Asimismo, las precondiciones (assert) atrapan correctamente referencias nulas y límites menores o iguales a cero (0, -1, -5).

Fallo de cursor JDBC en cargar(int id) (Rojo - CP-DAO-2.5 y CP-DAO-2.6):

El método DataAccessObject.cargar(int id) ejecuta la consulta parametrizada pero intenta leer directamente resultado.getInt("id") sin antes posicionar el cursor del ResultSet mediante resultado.next(). Como en JDBC el cursor inicia antes de la primera fila (*before first*), el método falla sistemáticamente lanzando SQLException: Before start of result set tanto para IDs existentes (1) como inexistentes (999).

Propagación del defecto de actualizar(Asociado) (Rojo - CP-DAO-3.1 y CP-DAO-3.2):

Debido a que DataAccessObject.actualizar delega en BaseDeDatos.ejecutarActualizacion (donde el WHERE id = ? está fijado estáticamente en 1), actualizar un asociado preexistente con id != 1 (como *Maria Template*, id = 2) provoca una SQLIntegrityConstraintViolationException al intentar asignarle al registro id = 1 un DNI que ya existe en el registro id = 2. Peor aún, si se actualiza un asociado cambiando también su DNI, el método sobrescribe silenciosamente al asociado id = 1 (*Juan State*) sin modificar al asociado real.

Asimetría de validaciones y corrupción en la lectura (Rojo - CP-DAO-4.4 y CP-DAO-4.5):



Mientras que eliminar(String dni) verifica !dni.isEmpty(), ni Asociado, ni AsociadoDTO, ni guardar(Asociado) validan cadenas vacías, permitiendo persistir registros completamente en blanco ("").

Finalmente, el caso CP-DAO-4.5 demuestra el riesgo de tener validaciones asimétricas entre la escritura y la lectura: como el constructor de AsociadoDTO no valida numero >= 0 pero el método setNumero sí lo hace (usado dentro de dao.CrearDTO al leer la base de datos), cualquier registro con número negativo almacenado en MySQL inutiliza por completo el método dao.cargarBD(), haciendo que toda la lectura de la tabla aborte con un AssertionError.

Modulo 2: Testing de la máquina de estados

Objetivo y Alcance de la Prueba

El objetivo de esta sección es validar el comportamiento de la clase Ambulancia bajo el patrón de diseño *State* (implementado a través de la interfaz State y sus clases concretas). Se busca garantizar que la máquina de estados gestione correctamente las transiciones ante los distintos eventos del sistema, aislando la lógica de transición de las dependencias externas y rutinas concurrentes mediante el uso de dobles de prueba (*Mocks*).

Estrategia de Testing y Uso de Dobles de Prueba (Mocks)

Dado que los métodos de transición de la clase Ambulancia requieren la interacción con actores del sistema (Asociado, Operario) y rutinas de ejecución concurrente (RetornoAutomatico), se aplicó una estrategia de aislamiento con Mockito:

Desacoplamiento de actores e hilos: Se instanciaron objetos mock para Asociado.class, Operario.class y RetornoAutomatico.class. Esto permite invocar las acciones sobre la ambulancia sin depender del estado interno de los actores ni disparar hilos de ejecución reales que introduzcan condiciones de carrera durante la verificación determinística de la tabla de estados.

Estructura AAA (Arrange-Act-Assert):

Arrange (@Before y precondición): Se instancia una Ambulancia limpia antes de cada test (setUp()) y se fuerza el estado inicial deseado mediante ambulancia.setState(...), creando el mock correspondiente al estímulo.

Act (Estímulo): Se invoca el método disparador en la ambulancia (atenderDomicilio, trasladarALaClinica, retornoAutomatico o solicitarMantenimiento).

Assert (Verificación): Se comprueba que la clase del estado resultante (ambulancia.getEstadoActual().getClass()) coincida exactamente con el estado esperado según la especificación.

Diseño de Casos de Prueba (Particiones de Equivalencia)

Para lograr una cobertura exhaustiva de transiciones (*All-Transitions Coverage*), se dividió el dominio de entrada en 6 particiones de equivalencia correspondientes a los estados posibles de la ambulancia, evaluando cada uno frente a los 4 eventos disparadores del sistema (conformando una matriz completa de 24 (6 x 4) casos de prueba).

Estado Inicial	Atención Domicilio	Traslado Clínica	Retorno Automático	Mantenimiento
Disponible	Atendiendo en Domicilio	Trasladando Paciente	Disponible	En Taller
Trasladando Paciente	Trasladando Paciente	Informa que no puede	Disponible	Informa que no puede
Atendiendo en Domicilio	Atendiendo en Domicilio	Informa que no puede	Regresando sin Paciente	Informa que no puede



Regresando sin Paciente	Atendiendo en Domicilio	Trasladando Paciente	Disponible	Informa que no puede
En Taller	En Taller	Informa que no puede	En Taller	Regresando del taller
Regresando del Taller	Regresando del Taller	Informa que no puede	Disponible	Informa que no puede

Detalle de Escenarios Implementados en testAmbulancia

Se definió un escenario de prueba por cada uno de los 6 estados en los que se puede encontrar la ambulancia, implementando 4 métodos @Test por escenario dentro de la clase testAmbulancia:

Escenario 1 (DisponibleState): Evalúa el inicio de operaciones desde el estado libre hacia atención domiciliaria, traslado a clínica, mantenimiento o permanencia ante retorno automático.

Escenario 2 (TrasladandoPacienteClinicaState): Evalúa la retención del estado durante un traslado activo y su liberación únicamente mediante el retorno automático.

Escenario 3 (AtendiendoEnDomicilioState): Evalúa la exclusividad durante la atención domiciliaria y la transición hacia el regreso sin paciente una vez finalizada.

Escenario 4 (RegresandoClinicaSinPacienteState): Evalúa la capacidad de reasignar la ambulancia en ruta ante nuevos pedidos de asociados, el rechazo de mantenimiento en viaje y la llegada definitiva a la clínica.

Escenario 5 (EnTallerState): Evalúa la inhabilitación de la unidad para usos médicos mientras está en reparación y el paso al regreso del taller ante la orden del operario.

Escenario 6 (RegresandoDelTallerState): Evalúa el bloqueo de solicitudes durante el trayecto de vuelta desde el taller hasta su arribo a la clínica (DisponibleState).

Mapeo entre la Tabla de Estados y las Aserciones (assertEquals):

Para conectar el diseño de los escenarios con la validación en código, en cada prueba se compara la clase del estado resultante (ambulancia.getEstadoActual().getClass()) contra la clase esperada aplicando dos criterios:

Transiciones efectivas: Cuando la tabla especifica un nuevo estado, se verifica que el objeto de estado interno de Ambulancia haya cambiado a la nueva instancia correspondiente.
Celdas de permanencia e "Informa que no puede": En aquellas intersecciones donde la tabla original indica que la unidad mantiene su estado o "Informa que no puede" atender la solicitud, la operación es rechazada a nivel de negocio sin alterar el ciclo de vida de la ambulancia. En el código de prueba, esto se valida comprobando que, tras ejecutar el método disparador, la clase del estado actual siga siendo exactamente la misma que antes del estímulo.

Batería de Casos de Prueba (testAmbulancia)

A partir de los 6 escenarios planteados, se ejecutó la batería de 24 casos de prueba utilizando dobles de prueba (mockAsociado, mockOperario y mockRetorno). En los casos donde la tabla del enunciado



especifica *"Informa que no puede"*, la salida esperada a nivel de máquina de estados es que la unidad rechace la transición y mantenga su estado inicial sin alteraciones. Es relevante mencionar que para la entrada atenderDomicilio y la entrada trasladarALaClínica siempre se envía mockAsociado, para la entrada retornoAutomatico se pasa un mock de RetornoAutomatico y para la solicitarMantenimiento un mock de la clase Operario.

Escenario 1: Ambulancia en estado DisponibleState Precondición: ambulancia.setState(new DisponibleState(ambulancia))

ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-1.1	atenderDomicilio()	AtendiendoEnDomicilioState	AtendiendoEnDomicilioState	PASA
CP-ME-1.2	trasladarALaClinica()	TrasladandoPacienteClinicaState	TrasladandoPacienteClinicaState	PASA
CP-ME-1.3	retornoAutomatico()	DisponibleState	DisponibleState	PASA
CP-ME-1.4	solicitarMantenimiento()	EnTallerState	EnTallerState	PASA

Escenario 2: Ambulancia en estado TrasladandoPacienteClinicaState
Precondición: ambulancia.setState(new TrasladandoPacienteClinicaState(ambulancia))

ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-2.1	atenderDomicilio()	TrasladandoPacienteClinicaState	TrasladandoPacienteClinicaState	PASA
CP-ME-2.2	trasladarALaClinica()	TrasladandoPacienteClinicaState	TrasladandoPacienteClinicaState	PASA
CP-ME-2.3	retornoAutomatico()	DisponibleState	RegresandoDelTallerState	FALLA
CP-ME-2.4	solicitarMantenimiento()	TrasladandoPacienteClinicaState	TrasladandoPacienteClinicaState	PASA

Escenario 3: Ambulancia en estado AtendiendoEnDomicilioState
Precondición: ambulancia.setState(new AtendiendoEnDomicilioState(ambulancia))

ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-3.1	atenderDomicilio()	AtendiendoEnDomicilioState	AtendiendoEnDomicilioState	PASA
CP-ME-3.2	trasladarALaClinica()	AtendiendoEnDomicilioState	AtendiendoEnDomicilioState	PASA
CP-ME-3.3	retornoAutomatico()	RegresandoClinicaSinPacienteState	RegresandoClinicaSinPacienteState	PASA
CP-ME-3.4	solicitarMantenimiento()	AtendiendoEnDomicilioState	AtendiendoEnDomicilioState	PASA

Escenario 4: Ambulancia en estado RegresandoClinicaSinPacienteState
Precondición: ambulancia.setState(new RegresandoClinicaSinPacienteState(ambulancia))



ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-4.1	atenderDomicilio()	AtendiendoEnDomicilioState	AtendiendoEnDomicilioState	PASA
CP-ME-4.2	trasladarALaClinica()	TrasladandoPacienteClinicaState	TrasladandoPacienteClinicaState	PASA
CP-ME-4.3	retornoAutomatico()	DisponibleState	DisponibleState	PASA
CP-ME-4.4	solicitarMantenimiento()	RegresandoClinicaSinPacienteState	RegresandoClinicaSinPacienteState	PASA

Escenario 5: Ambulancia en estado EnTallerState
Precondición: ambulancia.setState(new EnTallerState(ambulancia))

ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-5.1	atenderDomicilio()	EnTallerState	EnTallerState	PASA
CP-ME-5.2	trasladarALaClinica()	EnTallerState	EnTallerState	PASA
CP-ME-5.3	retornoAutomatico()	EnTallerState	EnTallerState	PASA
CP-ME-5.4	solicitarMantenimiento()	RegresandoDelTallerState	RegresandoDelTallerState	PASA

Escenario 6: Ambulancia en estado RegresandoDelTallerState
Precondición: ambulancia.setState(new RegresandoDelTallerState(ambulancia))

ID	Entrada (Evento)	Salida Esperada	Salida Obtenida	Resultado
CP-ME-6.1	atenderDomicilio()	RegresandoDelTallerState	RegresandoDelTallerState	PASA
CP-ME-6.2	trasladarALaClinica()	RegresandoDelTallerState	RegresandoDelTallerState	PASA
CP-ME-6.3	retornoAutomatico()	DisponibleState	DisponibleState	PASA
CP-ME-6.4	solicitarMantenimiento()	RegresandoDelTallerState	RegresandoDelTallerState	PASA

Resultados de Ejecución y Detección de Defectos (AmbulanciaTest):

Al ejecutar la suite conformada por los 24 casos de prueba (AmbulanciaTest / testAmbulancia) sobre el código provisto por la cátedra, se obtuvo el siguiente resultado general (Ver figura 2 en anexo):

Tests ejecutados: 24
Pruebas exitosas (En Verde): 23
Fallos de aserción (Failures - En Rojo): 1
Errores de ejecución (Errors): 0
Balance de Cobertura por Escenario

Escenarios validados exitosamente (Verde): Los Escenarios 1, 3, 4, 5 y 6 superaron el 100% de sus casos de prueba (4/4 cada uno), confirmando que sus clases de estado respetan tanto las transiciones válidas como los bloqueos especificados. Dentro del Escenario 2, las 3 pruebas correspondientes a solicitudes de asociados y de mantenimiento también resultaron exitosas.

Escenario con defecto detectado (Rojo): El único fallo de la suite se localizó en el Escenario 2 (TrasladandoPacienteClinicaState) ante el evento de retorno automático (ver figura 2 del anexo).

Análisis del Defecto Detectado

Caso de prueba fallido: testSolicitudAmbulanciaTrasladandoPacienteClinicaParaRetornoAutomatico (línea 102).

Traza del fallo: `AssertionError: Debe pasar a Disponible expected:<class modelo.modeloDominio.ambulancia.DisponibleState> but was:<class modelo.modeloDominio.ambulancia.RegresandoDelTallerState>.`

Diagnóstico y causa raíz: Según la tabla de especificación de estados, cuando la ambulancia se encuentra en Trasladando Paciente (TrasladandoPacienteClinicaState) y recibe el estímulo de Retorno Automático, debe finalizar el traslado en la clínica y transicionar al estado Disponible (DisponibleState). Sin embargo, la prueba demostró que la implementación provista de TrasladandoPacienteClinicaState contiene un defecto lógico en dicho método, instanciando erróneamente RegresandoDelTallerState. Esto provoca una inconsistencia grave en el modelo de negocio, ya que luego de dejar a un paciente en la clínica, la ambulancia queda registrada como si estuviera volviendo de una reparación mecánica en el taller.

Conclusiones del Módulo de Máquina de Estados

El diseño y ejecución de las pruebas sobre la máquina de estados de la clase Ambulancia permitió extraer las siguientes conclusiones técnicas:

Eficacia de la cobertura exhaustiva de transiciones: La sistematización de la matriz de 6 x 4 mediante particiones de equivalencia demostró ser indispensable. Un conjunto de pruebas parcial o enfocado solo en los caminos principales podría haber pasado por alto el desvío anómalo entre TrasladandoPacienteClinicaState y RegresandoDelTallerState.

Aislamiento mediante Mocks: El uso de dobles de prueba (Mockito.mock) para representar a Asociado, Operario y RetornoAutomatico permitió evaluar la lógica pura del patrón *State* de manera determinística, rápida y sin interferencias de los tiempos de espera ni de la planificación de hilos del sistema operativo, los cuales se delegan específicamente al Módulo 3 (Concurrencia).

Integridad de las restricciones de estado: Se comprobó que los estados bloqueantes y las situaciones donde la unidad *"Informa que no puede"* mantienen inmutable la referencia del estado actual en Ambulancia, garantizando que solicitudes inválidas no interrumpan traslados médicos ni tareas de mantenimiento en curso.

Modulo 3: Testing de las funcionalidades de concurrencia



El módulo de concurrencia del sistema cuenta con la clase Ambulancia, Asociado, Operario y RetornoAutomatico. Dichas clases interactúan de distintas maneras con el recurso compartido (Ambulancia):

Los Asociado solicita asistencia de la ambulancia.

El Operario intenta, periódicamente, mandar la ambulancia a mantenimiento.

RetornoAutomatico se encarga de asegurarse de que la ambulancia retorne a la clínica cada cierto tiempo.

El objetivo de esta sección es probar como interactúan los distintos hilos en ejecución, verificando la exclusión mutua del recurso compartido y el correcto funcionamiento de los bloques de sincronización. Es importante aclarar que todos los métodos de prueba incorporan la anotación @Test(timeout = ...) de JUnit. Si un error en las primitivas wait() o notifyAll() deja a un hilo atrapado indefinidamente en la cola de espera del monitor, el test falla por tiempo de expiración. Para esto se elaboraron los siguientes tests:

Caso de Prueba	Aspecto Validado	Descripción
testSegundoAsociadoEsperaYSeDespierta	Bloqueo por guarda (wait) y reactivación (notifyAll)	Un primer asociado ocupa la ambulancia y un segundo hilo intenta usarla, quedando en Thread.State.WAITING. Al liberarse con regresarSinPaciente(), el segundo hilo despierta, toma el recurso (isOcupado() == true) y transiciona a AtendiendoEnDomicilioState.
testNuncaUsanLaAmbulanciaDosALaVez	Exclusión mutua estricta bajo estrés	Se lanzan 10 hilos concurrentes ejecutando 20 ciclos de solicitud y liberación cada uno (200 accesos en total). Se verifica que el pico máximo de hilos simultáneos en la sección crítica sea exactamente 1 (maximo.get() == 1) y que al finalizar quede libre.
testFinalizarSimulacionLiberaHilosEnEspera	Cierre limpio de hilos bloqueados	Con la ambulancia ocupada y 3 hilos en estado WAITING, se invoca setSimulacionActiva(false). Se comprueba mediante join() (bajo un timeout de 5s) que todos los hilos en espera son notificados y finalizan su ejecución sin quedar colgados.
testRetornoAutomaticoLlamaALaAmbulancia	Integración del hilo RetornoAutomatico con mock de State	Se inyecta un mock(State.class) en la ambulancia y se inicia el hilo demonio RetornoAutomatico. Mediante

		verify(..., timeout(12000)), se valida que el hilo temporal invoque efectivamente el método RetornoClinica() sobre el estado tras su ciclo de espera.
testSimulacionCompletaTermina	Prueba de integración de sistema completo	Se instancian e inician hilos reales para 3 objetos Asociado (con 2 solicitudes cada uno) y 1 Operario compitiendo por la misma instancia de Ambulancia. Tras el join() de todos los hilos, se verifica que la simulación concluya (! isSimulacionActiva()) y el recurso quede liberado (! isOcupado()).

Tabla de Particiones de Equivalencia:

Condición de entrada	Clases Válidas (CEV)	Clases Inválidas (CEI)
1. Disponibilidad del Monitor (Ambulancia.ocupado)	CEV-1: ocupado == false (recurso libre: el hilo ingresa a la sección crítica y cambia ocupado = true). CEV-2: ocupado == true con isSimulacionActiva == true (recurso ocupado: el hilo se bloquea legítimamente en wait()).	CEI-1: Más de un hilo poseyendo el recurso simultáneamente (hilosEnSeccionCritica > 1, violación de exclusión mutua).
2. Cantidad de Hilos Competidores (N)	CEV-3: N > 0	CEI-2: N<=0 (sin hilos de simulación).
3. Ciclo de Vida de Simulación (isSimulacionActiva)	CEV-4: isSimulacionActiva == true (los hilos compiten normalmente y RetornoAutomatico cicla). CEV-5: Transición true -> false con hilos en WAITING (notifyAll() despierta a todos y abortan sin deadlock).	CEI-3: isSimulacionActiva == false al invocar un servicio (el hilo aborta inmediatamente en el if (! isSimulacionActiva()) return;).
4. Solicitudes de Asociado (cantSolicitudesAtendidas vs max)	CEV-9: 0≤atendidas<maxCantSolicitudes (el asociado sigue pidiendo ambulancia). CEV-10: atendidas==maxCantSolicitudes en todos los asociados	CEI-6: atendidas > maxCantSolicitudes (exceso de iteraciones).

	(verificarYFinalizarSimulacion() finaliza la simulación).	

Batería de Casos de Prueba:

ID	CE	Estímulo	Salida Esperada	Resultado
CP-CONC-01	CEV-1, 2, 4	regresarSinPaciente() + segundo.join()	segundo despierta y toma la ambulancia (AtendiendoEnDomicilioState, ocupada).	PASA
CP-CONC-02	CEV-1, 2, 4; CEI-1	10 hilos × 20 ciclos de tomar/liberar	maximo == 1 y al final no ocupada.	PASA
CP-CONC-03	CEV-2, 5; CEI-3;	setSimulacionActiva(false) + join(5000)	Los 3 hilos despiertan y terminan sin deadlock.	PASA
CP-CONC-04	CEV-4	start() + verify(estadoMock, timeout(12000))	A los 10 s se invoca estadoMock.RetornoClinica().	FALLA[1]
CP-CONC-05	CEV-1, 2, 4, 9, 10	start() de 4 hilos + join(120000)	La simulación finaliza: inactiva y no ocupada.	PASA

[1] RetornoAutomatico.java:25: la condición while (!ambulancia.isSimulacionActiva()) está negada e impide entrar al bucle.

Resultados de Ejecución y Detección de Defectos (AmbulanciaConcurrenciaTest):

Al ejecutar la batería de 5 casos de prueba de concurrencia (Ver figura 3 en anexo) e integración sobre el sistema provisto, se obtuvo el siguiente balance general:

Tests ejecutados: 5
Pruebas exitosas (Verde): 4 (CP-CONC-01, CP-CONC-02, CP-CONC-03 y CP-CONC-05)
Fallos de verificación (Failures - Estado Rojo): 1 (CP-CONC-04)
Interpretación de las pruebas exitosas (Estado Verde): Los resultados positivos en CP-CONC-01, CP-CONC-02, CP-CONC-03 y CP-CONC-05 demuestran que la estructura del monitor implementada dentro de la clase Ambulancia gestiona de forma segura el acceso concurrente a la sección crítica. El uso de bloques synchronized, la guarda de espera activa (while (isOcupado() && isSimulacionActiva())) con wait(), y la notificación mediante notifyAll() garantizan tanto la exclusión mutua estricta (maximo == 1 bajo estrés de 200 accesos concurrentes en CP-CONC-02) como la ausencia de bloqueos permanentes (*deadlocks*) al dar por finalizada la simulación (CP-CONC-03 y CP-CONC-05).

Análisis del defecto detectado (Estado Rojo en CP-CONC-04):

Caso de prueba fallido: testRetornoAutomaticoLlamaALaAmbulancia (CP-CONC-04).

Traza del fallo: Wanted but not invoked: state.RetornoClinica(); -> RetornoAutomatico debería hacer volver a la ambulancia a los 10 segundos, pero no lo hizo (Expiración del timeout(12000) de Mockito).



Diagnóstico y causa raíz: El hilo demonio RetornoAutomatico tiene la responsabilidad de invocar periódicamente el evento de retorno a la clínica sobre la ambulancia mientras la simulación se encuentre en curso. Sin embargo, al inspeccionar el método run() en RetornoAutomatico.java (línea 25), se detectó que la condición del bucle principal se encuentra lógicamente invertida: while (! ambulancia.isSimulacionActiva()).

Impacto en el sistema: Dado que la ambulancia se instancia con simulacionActiva = true, al iniciarse el hilo RetornoAutomatico la expresión !ambulancia.isSimulacionActiva() evalúa inmediatamente a false. En consecuencia, el hilo jamás ingresa al bucle de espera ni ejecuta ambulancia.retornoAutomatico(), finalizando su ejecución de manera prematura e impidiendo el retorno automático de las unidades en un escenario real. (La captura de ejecución correspondiente se adjunta en el Apéndice).

Conclusiones del Módulo de Concurrencia

El diseño y ejecución de la suite AmbulanciaConcurrenciaTest permitió evaluar los aspectos críticos de la programación multi-hilo sobre el recurso compartido Ambulancia, arribando a las siguientes conclusiones:

- *Garantía de Exclusión Mutua y Prevención de Condiciones de Carrera:* Mediante la instrumentación con variables atómicas (AtomicInteger) en pruebas de estrés (CP-CONC-02), se comprobó empíricamente que los métodos sincronizados de Ambulancia impiden que dos hilos Asociado tomen el recurso de forma simultánea (CEI-1), preservando la consistencia del estado interno frente a alta contención.

- *Correcta Gestión de Colas de Espera y Cierre Limpio (Graceful Shutdown):* Se verificó que los hilos que encuentran la ambulancia ocupada se suspenden correctamente en estado WAITING sin consumir ciclos de CPU, despertando en orden a medida que el recurso se libera (CP-CONC-01) o abortando su espera de manera segura cuando setSimulacionActiva(false) emite la señal de cierre global (CP-CONC-03 y CP-CONC-05), evitando *deadlocks* e inanición (*starvation*).

- *Detección de Fallos de Concurrencia mediante Mocks Asíncronos:* La combinación de un doble de prueba (mock(State.class)) con la verificación temporal verify(..., timeout(12000)) en CP-CONC-04 resultó determinante para aislar y descubrir el defecto en RetornoAutomatico. Mientras que en una prueba manual o de consola la muerte silenciosa del hilo temporizador podría pasar desapercibida, la aserción temporal expuso de forma directa la condición booleana negada (!ambulancia.isSimulacionActiva()) en el ciclo de vida del hilo.

Conclusión General del Informe

El desarrollo del presente trabajo práctico permitió aplicar de manera integral las técnicas y conceptos fundamentales del testing de software en la auditoría de un sistema complejo de gestión clínica. A lo largo de las distintas etapas de evaluación, se estructuró y ejecutó un enfoque de validación diferenciado en dos niveles arquitectónicos principales: el testing atómico y el testing de integración con componentes externos y concurrencia.

Eficiencia de la Combinación de Técnicas de Prueba:

La aplicación de particiones de equivalencia y análisis de valores límite en los componentes unitarios (AsociadoDTO) permitió exponer defectos de valores limites, tal como se evidenció en la validación numérica de la altura del domicilio.
El uso de patrones de aislamiento con Mocks en la máquina de estados de la Ambulancia facilitó la cobertura exhaustiva de transiciones (6 x 4), aislando la lógica de negocio de los actores externos y detectando anomalías puntuales en la matriz de estados.
Las pruebas de integración contra MySQL (BaseDeDatos y DataAccessObject) demostraron que el acoplamiento con persistencia real es indispensable para descubrir vulnerabilidades que no se manifiestan en memoria, como el desvío en los datos semilla, fallos de cursor JDBC y errores de indexación en actualizaciones críticas.
Finalmente, el acoplamiento de hilos de ejecución concurrentes y monitores permitió auditar la exclusión mutua bajo estrés y el comportamiento de las colas de espera, validando la estabilidad operativa del sistema ante múltiples usuarios simultáneos.

Valor de la Detección Temprana de Fallos:

La sistematización de las pruebas en suites automatizadas (JUnit) evidenció cómo la ejecución rigurosa bajo escenarios controlados (tanto en estado *Rojo* como *Verde*) cumple el propósito fundamental de asegurar la calidad del producto: no solo certificar el funcionamiento de los caminos felices, sino descubrir de forma temprana inconsistencias de sincronización, excepciones de SQL no controladas y desvíos lógicos en los componentes acoplados antes de su paso a producción.

Tabla global de resultados:

Módulo	Suite	Casos	PASA	FALLA
AsociadoDTO	AsociadoDTOTest	8	7	1
BaseDeDatos	BaseDeDatosTest	17	13	4
DataAccessObject	DataAccessObjectTest	18	12	6
Máquina de estados	AmbulanciaTest	24	23	1
Concurrencia	AmbulanciaConcurrenciaTest	5	4	1



Total		**72**	**59**	**13**

Tabla consolidada de defectos

Componente	Descripción	Casos que lo evidencian
`BaseDeDatos.ejecutarActualizacion`	`stmt.setInt(8, 1)` hardcodeado: siempre modifica el registro id = 1	CP-BD-3.3, CP-DAO-3.1, CP-DAO-3.2
`DataAccessObject.cargar(int)`	Lee el `ResultSet` sin llamar antes a `resultado.next()`	CP-DAO-2.5, CP-DAO-2.6
Constructor de `AsociadoDTO` y DDL de `Asociados`	No validan nulos ni `numero >= 0`; un registro inválido se persiste y luego rompe `cargarBD()`	CP-BD-4.6, CP-DAO-4.5
`DataAccessObject.guardar`	Acepta y persiste cadenas vacías	CP-DAO-4.4
`AsociadoDTO.setNumero`	El `assert` usa `> 0` en lugar de `>= 0` y rechaza el valor válido 0	CP-DTO-6.1
`TrasladandoPacienteClinicaState`	Ante el retorno automático pasa a `RegresandoDelTallerState` en vez de `DisponibleState`	CP-ME-2.3
`RetornoAutomatico.run()`	Condición del `while` negada: el hilo nunca entra al bucle	CP-CONC-04
Datos semilla (`iniciarBD`)	El registro id = 4 tiene un espacio inicial en `ciudad`	CP-BD-2.4
Datos semilla (`iniciarBD`)	Los registros id = 12 y 17 tienen teléfonos de 9 dígitos	CP-BD-2.5

Anexo

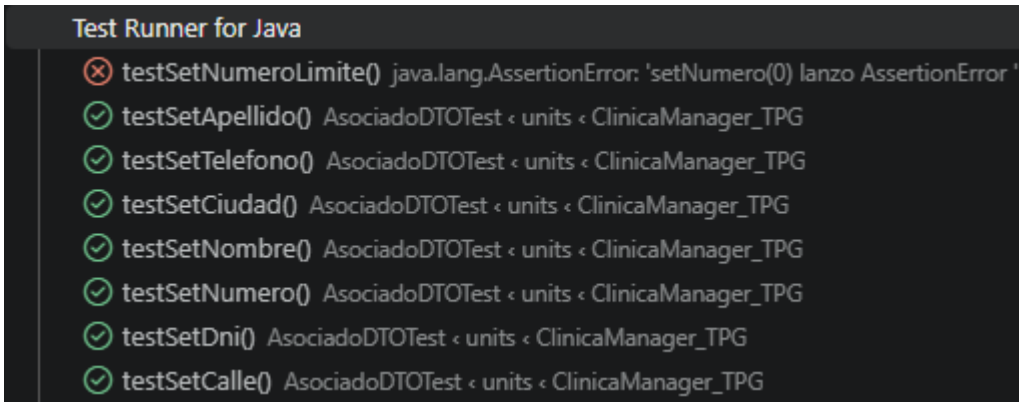


Figura 1: Ejecución de la suite de pruebas unitarias para AsociadoDTO (AsociadoDTOTest), evidenciando 7 pruebas exitosas y el fallo por AssertionError en el análisis del valor límite con testSetNumeroLimite().

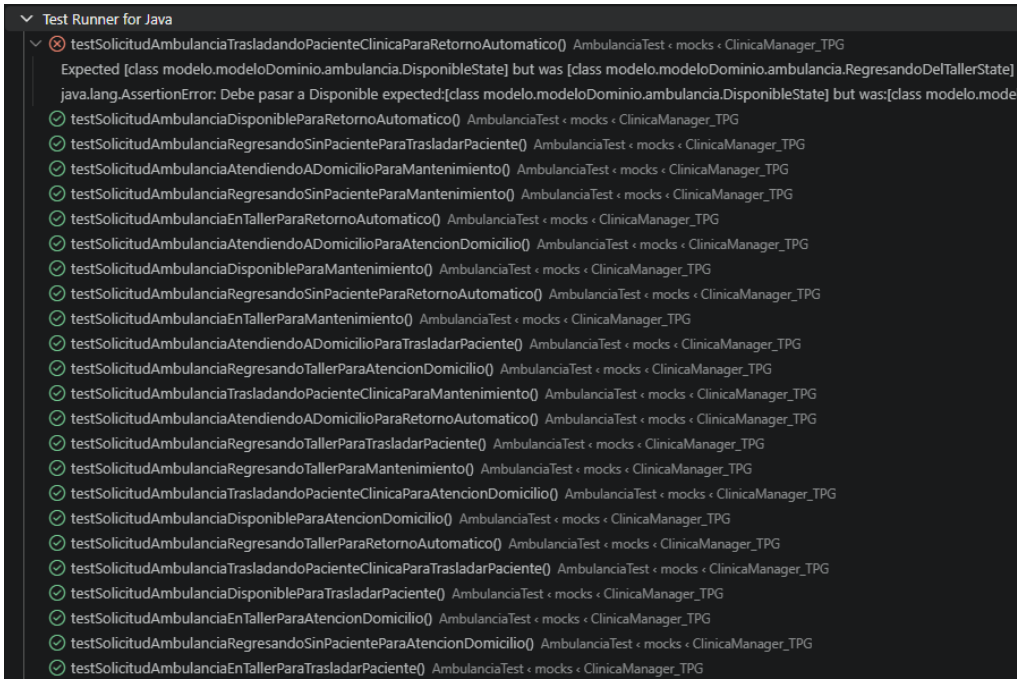


Figura 2: Resultados de la suite de pruebas de la máquina de estados (AmbulanciaTest), mostrando 23 transiciones exitosas y el fallo detectado en testSolicitudAmbulanciaTrasladandoPacienteClinicaParaRetornoAutomatico.

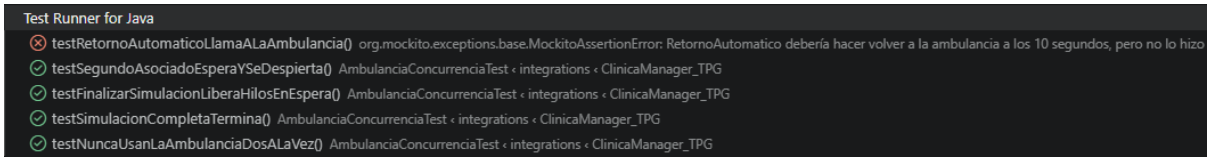


Figura 3: Ejecución de la suite de integración y concurrencia (AmbulanciaConcurrenciaTest), reflejando la correcta validación multi-hilo de los bloques sincronizados junto al fallo temporal detectado en testRetornoAutomaticoLlamaALaAmbulancia.

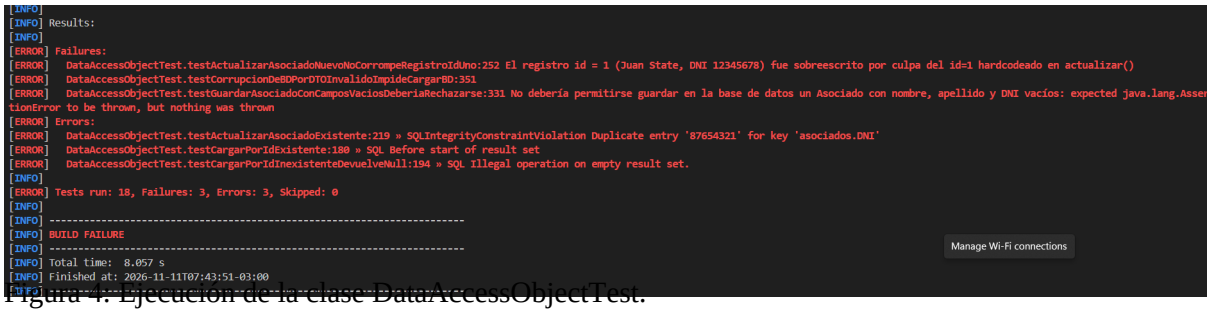


Figura 4: Ejecución de la clase DataAccessObjectTest.